Design of a Low-Power, Secure, Customized PQC Single-Core Processor Based on the RISC-V ISA

Authors: Quentin TRIPARD, Marianne NINET, Xinda GUO, Noé VERDEYME
IMT Atlantique - Brest, France

Supervisors: Yehya NASSER, Magali LE GALL
MEE Department, Low Power Secure Silicon (LPSS) Group

PROCOM Project 2025/2026

This paper presents the design and implementation of a secure, low-power RISC-V processor optimized for Post-Quantum Cryptography (PQC) algorithms. With the emergence of quantum computing threatening current cryptographic standards such as RSA and ECC, implementing resource-efficient PQC solutions on embedded systems becomes critical. We focus on the HQC (Hamming Quasi-Cyclic) algorithm, analyzing its computational bottlenecks through profiling on the Gem5 simulator and implementing custom RISC-V instructions to accelerate critical operations. Our approach combines software profiling, hardware implementation on FPGA using the Pulpino core deployed on the ChipWhisperer CW305 board (Xilinx Artix-7), and security evaluation through side-channel attack analysis. We demonstrate a 33 percent performance improvement through the implementation of a custom carry-less multiplication (CLMUL) instruction targeting the Karatsuba multiplication bottleneck. Additionally, we validate our side-channel analysis methodology by successfully performing a Correlation Power Analysis (CPA) attack on AES-128, recovering the complete key with only 200 power traces.

Keywords: PQC, RISC-V, CPU Microarchitecture, Low Power, Security

1 Introduction

The advent of quantum computing poses a significant threat to current cryptographic standards. Shor's algorithm can efficiently factor large integers and compute discrete logarithms, rendering RSA and ECC-based cryptosystems vulnerable to quantum attacks. This necessitates the transition to Post-Quantum Cryptography (PQC) algorithms that resist both classical and quantum attacks.

However, PQC algorithms are computationally intensive and require substantial memory resources, making their deployment on resource-constrained embedded systems challenging. The NIST standardization process has validated several candidates, including lattice-based schemes (CRYSTALS-Kyber, CRYSTALS-Dilithium), hash-based schemes (SPHINCS+), and code-based schemes (HQC).

This project addresses the challenge of implementing efficient and secure PQC on embedded RISC-V platforms. The RISC-V architecture provides a unique advantage, enabling us to design custom instructions tailored to the computational demands of PQC. Our objectives are threefold: (1)Implement PQC algorithms on a RISC-V architecture, (2)Optimize performance through custom instructions and hardware accelerators, and (3)Evaluate security against side-channel attacks.

We selected HQC (Hamming Quasi-Cyclic) as our target algorithm for several reasons: it operates on binary polynomials with simple operations (XOR for addition, Karatsuba algorithm for multiplication), well-suited for hardware acceleration. Moreover, existing literature on side-channel key-recovery attacks against HQC provides a good foundation for reproducing attack techniques and validating our experimental setup.

2 Technical Architecture

Our development environment consists of three main components working in a software-to-hardware validation pipeline.

2.1 Gem5 Simulator

Gem5 serves as our primary development and profiling platform. We configured it to simulate a RISC-V RV32IMC core with a MinorCPU model featuring a 4-stage pipeline (IF→ID→EX→WB), matching the Pulpino architecture. This configuration allows accurate cycle-level profiling and identification of computational bottlenecks before hardware deployment.

The simulator enables us to execute HQC implementations compiled for RISC-V using the PQClean library,

collect detailed instruction-level statistics and function-level profiling, simulate custom instruction behavior before RTL implementation, and compare RV32I versus RV32IMC performance characteristics.

2.2 Pulpino RISC-V Core

Pulpino is an open-source 32-bit RISC-V microcontroller developed by ETH Zurich (PULP platform). We deploy it on the ChipWhisperer CW305 target board featuring a Xilinx Artix-7 FPGA. The core implements the RV32IMC instruction set with a 4-stage pipeline and supports custom instruction extensions through available opcodes.

The Pulpino SoC architecture includes a RISC-V core with instruction and data RAM, AXI4 interconnect for memory access, APB bridge for peripheral communication, debug unit with JTAG interface, and SPI, UART, GPIO, and Timer peripherals.

A critical challenge we encountered is memory limitation. The default Pulpino configuration provides insufficient RAM for HQC's large polynomial operations. Our profiling revealed that HQC-128 requires approximately 42 KB of memory (Table 1).

Table 1: HQC Memory Requirements

text	data	bss	dec	hex
40530	1364	808	42702	a6ce

We are currently testing a way to get RAM extension, either through software optimization or physical RAM addition to the board.

2.3 ChipWhisperer Platform

The ChipWhisperer CW305 serves as a side-channel analysis tool. It provides high-resolution power measurement capability via dedicated shunt resistor, synchronized trigger mechanisms for trace alignment, Python API for automated trace acquisition and analysis, and direct access to FPGA power rails for accurate measurements.

3 AES Implementation and CPA Validation

Before attacking HQC, we validated our side-channel analysis methodology using AES-128 as a reference target.

3.1 AES-128 Implementation

We implemented AES-128 encryption on the Pulpino core. The algorithm consists of an initial AddRoundKey operation, followed by 9 main rounds (SubBytes, ShiftRows, MixColumns, AddRoundKey) and a final round without MixColumns. Each round processes a 128-bit state matrix organized as a 4×4 byte array.

The SubBytes operation applies a non-linear S-Box transformation to each byte, creating the data-dependent power consumption that enables CPA attacks. This non-linearity is essential for the attack as it creates distinguishable power patterns for different key hypotheses. We successfully acquired power traces during AES execution, with clear visibility of the 10 encryption rounds in the power consumption pattern.

3.2 Correlation Power Analysis Theory

CPA exploits the correlation between intermediate values during cryptographic computation and the device's power consumption. The attack model assumes a linear relationship between power consumption P and processed data D:

$$P = \alpha \cdot D + \beta + \epsilon \tag{1}$$

where α represents the power coefficient, β is a constant offset, and ϵ represents measurement noise.

For AES, we target the S-Box output after the first AddRoundKey:

$$V = \text{SBOX}(P_{\text{plaintext}}[i] \oplus K) \tag{2}$$

We model power consumption using the Hamming Weight model:

$$H(V) = \text{HW}(V) \tag{3}$$

The correlation coefficient between measured power P and hypothetical power model H is computed as:

$$\rho(P, H) = \frac{\text{cov}(P, H)}{\sigma_P \cdot \sigma_H} \tag{4}$$

The correct key byte hypothesis maximizes this correlation coefficient across all 256 possible values.

3.3 Experimental Results

We successfully recovered the complete 128-bit AES key using only 200 power traces. For each of the 16 key bytes, we computed correlation coefficients for all 256 possible key hypotheses across all time samples. The correct key hypothesis consistently showed the highest correlation peak. The recovered key bytes are shown in Table 2.

Table 2: Recovered AES-128 Key Bytes

Byte	Value	Byte	Value
0	0x2b	8	0xab
1	0x7e	9	0xf7
2	0x15	10	0x15
3	0x16	11	0x21
4	0x28	12	0x09
5	0xae	13	0xcf
6	0xd2	14	0x4f
7	0x99	15	0x3c

This matches the known test key: `2b7e151628aed2a6abf7158809cf4f3c`, validating our attack methodology and measurement setup.

4 HQC Algorithm Analysis

4.1 Algorithm Selection

A comparative overview of the evaluated PQC candidates is provided in Appendix **??** for clarity.

We selected HQC for its easier arithmetic operations (binary polynomial operations) and the documented vulnerabilities that we can reproduce to validate our attack methodology.

4.2 HQC Algorithm Overview

HQC is a code-based Key Encapsulation Mechanism (KEM) selected as an alternate candidate in NIST's PQC standardization. It relies on the hardness of decoding random quasi-cyclic codes.

The algorithm operates on binary polynomials in $\mathbb{F}_2[X]/(X^n - 1)$. Key operations include addition (XOR operation on polynomial coefficients), multiplication (Karatsuba algorithm for efficient polynomial multiplication), and error correction (BCH and Reed-Muller decoding).

The decapsulation process, which is our attack target, computes:

$$W = V \oplus (U \cdot Y) \tag{5}$$

where U and V are ciphertext components, and Y is part of the secret key.

4.3 Performance Profiling

We profiled HQC-128 execution on Gem5, collecting detailed statistics over 26,910,883 CPU cycles. The analysis revealed clear computational bottlenecks (Table 3).

Table 3: Top Functions by Instruction Count

Function	Instructions	%
karatsuba	7,324,011	84.36
vect_set_random	705,190	8.12
KeccakF1600	413,532	4.76
karatsuba_add2	124,988	1.44
karatsuba_add1	46,072	0.53

The Karatsuba multiplication function dominates execution time at 84.36%, making it the primary optimization target.

4.4 Instruction Distribution

Analysis of the dynamic instruction profile revealed the most frequently executed RISC-V instructions (Table 4).

Table 4: Top Executed Instructions

Instruction	Exec. ($\times 10^6$)	%
sub	1.46	16.8
c.lw	1.28	14.8
and	0.94	10.8
xor	0.63	7.3
or	0.53	6.1
sll	0.52	6.0
c.srli	0.47	5.4
c.addi	0.39	4.5

The prevalence of shift and XOR operations in the Karatsuba implementation motivated our choice of custom instruction.

5 Custom Instruction Implementation

5.1 Optimization Analysis

We identified several optimization candidates through pattern analysis (Table 5).

Table 5: Optimization Candidates

Pattern	Occur.	Gain	Solution
Soft-CLMUL	72,702	∼65k	clmul
Soft-Rotate	1,872	∼1.5k	ror/rol
Soft-Popcount	0	∼0	cpop
Vector Add	9,248	∼18k	SIMD

The carry-less multiplication (CLMUL) instruction emerged as the optimal choice, offering the best balance between implementation complexity and performance improvement.

5.2 Karatsuba Algorithm

The Karatsuba algorithm performs polynomial multiplication recursively. For input polynomials A and B on $t = 2^r$ computer words:

1. Split: $A = A_0 + x^{64t/2} \cdot A_1$ and $B = B_0 + x^{64t/2} \cdot B_1$
2. Recursive multiplication:

$$R_0 \leftarrow \text{Karatsuba}(A_0, B_0, t/2) \tag{6}$$
$$R_1 \leftarrow \text{Karatsuba}(A_1, B_1, t/2) \tag{7}$$
$$R_2 \leftarrow \text{Karatsuba}(A_0 + A_1, B_0 + B_1, t/2) \tag{8}$$

3. Reconstruction: $R \leftarrow R_0 + (R_0 + R_1 + R_2) \cdot X^{64t/2} + R_1 \cdot X^{64t}$

The base case ($t = 1$) performs a 64-bit carry-less multiplication, which is the bottleneck we target.

5.3 CLMUL Instruction Design

Carry-less multiplication (polynomial multiplication over GF(2)) computes the product of two polynomials where coefficients are in $\mathbb{F}_2$. Unlike standard multiplication, there are no carries between bit positions, additions are performed modulo 2 (XOR).

For 32-bit operands, the 64-bit result requires two instructions on RV32:

- `clmul_lo`: Returns bits [31:0] of the result
- `clmul_hi`: Returns bits [63:32] of the result

However, our first implementation ended up re-deriving the full 64-bit CLMUL product twice when software needs both halves (e.g., executing clmul_lo followed immediately by clmul_hi on the same operands).

To mitigate this, we add a small micro-architectural "CLMUL result cache" inside the multiplier:

When a clmul_* instruction is accepted, the multiplier stores the full 64-bit product P = clmul(a,b) along with the corresponding operands (a,b) and a valid bit.

On a subsequent clmul_lo clmul_hi, if the current operands match the cached operands, the unit returns the requested slice directly from the cached 64-bit product (P[31:0] or P[63:32]) instead of relying on a fresh internal recomputation.

The cached value persists until it is overwritten by the next accepted CLMUL operation or cleared by reset.

5.4 Instruction Encoding

RISC-V reserves four opcodes for custom instructions. Since Pulpino uses two of these, we utilized an available `funct7` field within an existing Pulpino opcode. The instruction follows R-type encoding with `funct3` differentiating between `clmul_lo` (funct3=0) and `clmul_hi` (funct3=1).

5.5 Hardware Implementation

The CLMUL unit implements Karatsuba decomposition at the hardware level. The 16-bit primitive function:

```
function automatic [31:0] clmul16;
  input logic [15:0] a;
  input logic [15:0] b;
  integer i;
  begin
    clmul16 = 32'b0;
    for (i = 0; i < 16; i++)
      if (b[i])
        clmul16 ^= (a << i);
  end
endfunction
```

The full 32×32→64 bit multiplication:

```
// Split operands into 16-bit halves
assign a_lo = op_a_i[15:0];
assign a_hi = op_a_i[31:16];
assign b_lo = op_b_i[15:0];
assign b_hi = op_b_i[31:16];

// Karatsuba over GF(2)
assign z0 = clmul16(a_lo, b_lo);
assign z2 = clmul16(a_hi, b_hi);
assign z1 = clmul16(a_lo ^ a_hi,
            b_lo ^ b_hi) ^ z0 ^ z2;

// Recomposition
assign clmul_full = {z2, 32'b0} ^
                    {z1, 16'b0} ^ z0;
assign clmul_lo = clmul_full[31:0];
assign clmul_hi = clmul_full[63:32];
```

5.6 Performance Results

Simulation results demonstrate significant improvement (Table 6).

Table 6: Performance Comparison

Metric	Baseline	CLMUL
simTicks	269.1B	181.4B
CPU Cycles	26.9M	18.1M
Instructions	40.8M	22.9M
simSeconds	0.269	0.181

The custom instruction achieves a **33% reduction in execution time**, validating our optimization approach.

5.7 Implementation Status

Current status: RTL Analysis in Vivado complete, bitstream generation complete, hardware testing in progress.

Potential concerns: (1) The combinational implementation may impact timing closure, and (2) the 64-bit result requires computing the full multiplication twice, suggesting optimization through result caching.

6 Side-Channel Attack Strategy for HQC

6.1 Attack Surface Analysis

We identified three potential attack vectors targeting HQC decapsulation (Table 7).

Table 7: HQC Attack Strategies

Attack	Target	Technique	Goal
1	BCH Dec.	Oracle	Partial key
2	Hadamard	Oracle	Full key
3	W comp.	DPA	Full key

6.2 Chosen-Ciphertext DPA Attack

Our primary attack targets $W = V \oplus (U \cdot Y)$ during decapsulation:

1. Construct ciphertexts (u, v) where u has a known structure
2. Send multiple ciphertexts with same u but varying v

3. Measure power consumption during decapsulation
4. Use DPA to correlate power with secret key Y bits
5. Each iteration leaks 64 bits of Y
6. Repeat until complete key recovery

6.3 Countermeasures

Countermeasures to evaluate include masking (introduce randomness into intermediate computations) and constant-time implementation (ensure execution time is independent of secret data).

Masking: Boolean masking splits sensitive variables into shares such that $x = x_1 \oplus x_2 \oplus ... \oplus x_d$ where d is the masking order. All operations must be performed on shares, preventing direct correlation between power consumption and secret values. For HQC, the polynomial multiplication $U \cdot Y$ must be masked, requiring careful implementation of masked XOR and shift operations.

Constant-time execution: Branch instructions dependent on secret data create timing variations exploitable by attackers. The BCH decoder in HQC is particularly vulnerable as syndrome computation and error correction exhibit data-dependent timing. Implementing constant-time BCH decoding requires replacing conditional branches with constant-time selection operations using bitwise masks.

Shuffling: Randomizing the order of independent operations across multiple executions prevents attackers from aligning traces at critical points. For HQC, the order of polynomial coefficient processing can be shuffled without affecting correctness.

7 Conclusion

We presented the design and partial implementation of a secure, low-power RISC-V processor optimized for post-quantum cryptography. Our main contributions include:

1. **Profiling**: HQC-128 on Gem5, identifying Karatsuba multiplication as the dominant bottleneck (84.36%)
2. **Custom instruction**: CLMUL achieving 33% performance improvement
3. **Side-channel validation**: CPA on AES-128 with 200 traces

Current work: Adding external RAM (Jan. 2026), implementing HQC on FPGA (Feb. 2026), testing CLMUL instruction.

Future work: DPA attack on HQC, countermeasure evaluation, final optimization (Feb.-Mar. 2026).

This work contributes to practical PQC deployment on resource-constrained embedded systems, addressing both performance and security requirements for IoT applications.

Acknowledgments

This work was conducted as part of the PROCOM project at IMT Atlantique under supervision of Dr. Yehya Nasser and Mrs. Magali Le Gall from the LPSS group.

References

[1] C. Aguilar-Melchor *et al.*, "HQC," NIST PQC Round 4, 2023.

[2] A. Waterman, K. Asanović, "RISC-V ISA Manual," v2.2, 2019.

[3] A. Traber *et al.*, "PULPino," ETH Zurich.

[4] N. Binkert *et al.*, "gem5 simulator," SIGARCH, 2011.

[5] C. O'Flynn, Z. Chen, "ChipWhisperer," COSADE 2014.

[6] E. Brier *et al.*, "CPA with leakage model," CHES 2004.

[7] A. Karatsuba, Y. Ofman, "Multiplication," Soviet Phys., 1963.

[8] S. Gueron, "Intel CLMUL," Intel, 2010.

[9] NIST, "PQC Standardization," csrc.nist.gov.

[10] PQClean Project, github.com/PQClean.

A PQC Candidate Comparison

Table 8: Comparison of NIST PQC Candidates

Algorithm	Type	Pros	Cons	Attacks
CRYSTALS-Kyber	Key Encap.	NIST standard	Module-LWE, heavy NTT	Yes
CRYSTALS-Dilithium	Signature	NIST standard	Module-LWE, Fiat-Shamir	Limited
SPHINCS+	Signature	Hash-based only	Large signatures	Limited
FALCON	Signature	Small signatures	Complex lattice theory	Limited
HQC	**Key Encap.**	**Binary poly., XOR, Karatsuba**	**Code-based**	**Yes**